

PC 603 CS — DISTRIBUTED SYSTEMS

Question Bank: 2 Marks & 6 Marks Questions with Answers

PC 603 CS — DISTRIBUTED SYSTEMS

Question Bank (2 Marks & 6 Marks Questions with Answers)

B.E. AI/ML — Osmania University Prepared with reference to the prescribed textbooks: Maarten van Steen & Andrew S. Tanenbaum, “Distributed Systems” (PHI, 4th Ed., 2023); Martin Kleppmann, “Designing Data-Intensive Applications” (O’Reilly, 2017); and the course syllabus.

UNIT – I: Introduction, Architectures, Processes & Communication

2 Marks Questions

Q1. What is a Distributed System? A distributed system is a collection of independent, autonomous computers that appears to its users as a single coherent system. The components communicate and coordinate their actions only through message passing over a network, and the goal is to present a unified, transparent view of resources and services to the user.

Q2. List any four design goals of a distributed system. (1) Resource sharing — making hardware, data, and services easily accessible; (2) Distribution transparency — hiding the fact that resources/processes are distributed across multiple machines; (3) Openness — offering services according to standard rules describing syntax and semantics; (4) Scalability — the ability to remain effective as the number of users, resources, or geographic spread grows.

Q3. What is distribution transparency? Name any three types. Distribution transparency is the property that hides the distributed nature of a system from users, making it appear as a single system. Types include: Access transparency (hides differences in data representation/access methods), Location transparency (hides where a resource is physically located), and Replication transparency (hides the fact that multiple copies of a resource exist).

Q4. What are the common pitfalls when building distributed systems? False assumptions commonly made by first-time distributed application developers include: the network is reliable, the network is secure, the network is homogeneous, latency is zero, bandwidth is infinite, transport cost is zero, and the topology does not change.

Q5. What is middleware in the context of distributed systems? Middleware is a software layer that sits between the underlying networked operating systems and distributed applications, providing common services (such as communication, naming, persistence, and

transactions) that make it easier to build distributed applications on top of heterogeneous platforms.

Q6. Differentiate between layered and symmetrically distributed (peer-to-peer) system architectures. In a layered architecture, components are organized in layers, with a component in a higher layer calling on services of components in the layer directly below it (e.g., client-server systems, multi-tier architectures). In a symmetrically distributed (peer-to-peer) architecture, all processes (peers) play similar roles, communicating as equals without a strict client-server hierarchy, often self-organizing (e.g., overlay networks like DHTs).

Q7. What is a hybrid system architecture? Give an example. A hybrid architecture combines elements of both layered/client-server and peer-to-peer architectures. An example is the Edge-server architecture used in Content Delivery Networks (CDNs), where edge servers cache content close to clients (layered) while cooperating with each other in a peer-to-peer fashion for content distribution.

Q8. What is a thread, and why are threads used in distributed systems servers? A thread is a lightweight unit of execution within a process that shares the process's memory/resources but has its own program counter and stack. Threads are used in servers to handle multiple client requests concurrently without the overhead of creating a new process for each request, improving performance and responsiveness.

Q9. What is virtualization, and why is it important in distributed systems? Virtualization is the technique of abstracting the hardware/software interface so that multiple virtual instances (virtual machines) can run independently on a single physical machine. It is important because it enables flexible, isolated resource allocation, easier system management, and improved fault isolation — forming the foundation of cloud computing.

Q10. What is code migration? Code migration is the ability to move code (and possibly its execution state) from one machine to another at runtime. It includes weak mobility (transferring only the program code) and strong mobility (transferring both code and execution state), and is used to improve performance by moving computation closer to data.

Q11. What is Remote Procedure Call (RPC)? RPC is a communication mechanism that allows a program to call a procedure located on a remote machine as if it were a local procedure call, hiding the underlying network communication details (marshalling parameters, sending messages, and unmarshalling results) from the programmer.

Q12. Differentiate between Message-Oriented and Stream-Oriented communication. Message-oriented communication involves discrete, self-contained messages exchanged between processes (e.g., via message queues), often supporting asynchronous, persistent communication. Stream-oriented communication involves a continuous flow of data (e.g., audio/video streaming) where the timing relationship between data items matters, requiring guarantees on data rate and synchronization rather than just delivery.

Q13. What is Multicast Communication? Multicast communication is the delivery of a single message from one sender to multiple receivers simultaneously (a subset of all possible receivers), as opposed to unicast (one-to-one) or broadcast (one-to-all).

6 Marks Questions

Q1. Explain the design goals of a Distributed System with suitable examples.

Distributed systems are built to satisfy several key design goals:

1. **Making resources accessible (Resource Sharing):** The primary goal is to let users and applications easily access and share remote resources — e.g., printers, storage, databases — as though they were local. Example: a networked file system (NFS) lets a user on one machine access files stored on a remote server transparently.
2. **Distribution Transparency:** The system should hide the fact that its processes and resources are physically distributed across multiple, possibly geographically separated, computers. Sub-types include access transparency, location transparency, migration transparency, relocation transparency, replication transparency, concurrency transparency, and failure transparency. Example: a cloud storage service like Google Drive hides which specific data center holds a user's file.
3. **Openness:** A distributed system should offer services according to standard rules (well-defined interfaces, protocols) that describe the syntax and semantics of those services, enabling interoperability between components built by different developers/vendors. Example: systems built on open standards like HTTP, REST APIs, or gRPC.
4. **Scalability:** The system must remain effective as the number of users, the size of data, or the geographical distribution of resources grows. Scalability includes size scalability (adding more users/resources), geographical scalability (functioning well despite large distances), and administrative scalability (spanning multiple independent administrative domains). Example: large web services like YouTube must scale to handle millions of concurrent users worldwide.

Achieving all these goals simultaneously involves trade-offs — for instance, achieving strong distribution transparency and consistency can conflict with scalability and performance, which is a recurring theme throughout distributed systems design (as later reflected in the CAP theorem).

Q2. Explain the different architectural styles used in distributed systems.

Architectural styles describe how the components of a distributed system are organized and how they interact:

1. **Layered Architectures:** Components are organized as a stack of layers, where a component in layer L_k calls components in layer L_{k-1} and provides services to layer L_{k+1} . Client-server systems and multi-tiered architectures (presentation, application/business logic, and data layers) are common examples — e.g., a typical three-tier web application with a browser (presentation), application server (logic), and database server (data).
2. **Object-Based Architectures:** Each component is treated as a separate “object,” and components communicate by invoking methods on each other, typically through RPC-

like mechanisms (e.g., CORBA, Java RMI). This offers flexibility, as any object can, in principle, communicate with any other.

3. **Data-Centered/Publish-Subscribe Architectures:** Components communicate indirectly through a shared data space or event bus rather than directly with each other; producers publish data/events and consumers subscribe to the data/events they are interested in (e.g., message brokers like Kafka, tuple spaces).
4. **Event-Based Architectures:** Closely related to publish-subscribe; processes communicate through the propagation of events, often combined with a shared data space (event-based + data-centered = “shared dataspace” architecture).

These architectural styles are combined in practice: **middleware organizations** apply one or more of these styles to structure real systems, while at a higher level, systems may be organized in **layered system architectures** (simple client-server, multi-tiered) or **symmetrically distributed architectures** (structured/unstructured peer-to-peer, e.g., DHT-based systems like Chord), or a **hybrid** of the two (e.g., edge-server systems, or BitTorrent-like systems that combine centralized trackers with peer-to-peer data transfer).

Q3. Explain the roles of Clients, Servers, and the concept of Code Migration in distributed systems.

A **server** is a process implementing a specific service (e.g., a file service, database service) that listens for and responds to requests from clients. A **client** is a process that requests a service from a server, typically by sending a request message and waiting for a response. This client-server model is the most common and simplest layered architecture, and can be extended to multi-tiered systems where an application server itself acts as a client to a database server.

Servers can be organized in different ways: an **iterative server** handles requests itself one at a time, while a **concurrent server** spawns a new thread/process to handle each incoming request, allowing multiple requests to be processed simultaneously. Servers can also be **stateless** (retain no client-specific information between requests, simplifying fault recovery, e.g., a simple web server) or **stateful** (maintain session/client information across requests, e.g., a file server tracking open file handles).

Code migration refers to moving executable code (and optionally its execution state) between machines at runtime, rather than only moving data. It is motivated by performance and flexibility — e.g., moving a computation to where the data resides rather than transferring large volumes of data across the network (“bringing the computation to the data”). Two forms exist: **weak mobility**, where only the program code (and possibly initialization data) is transferred, and execution restarts from the beginning on the target machine; and **strong mobility**, where both the code and its current execution state (e.g., stack, program counter) are transferred, allowing execution to resume exactly where it left off. Mobile agents and some Java-based migration frameworks are classic examples of this concept.

Q4. Explain Remote Procedure Call (RPC) with its working mechanism and limitations.

RPC allows a client process to invoke a procedure that executes on a remote server, giving programmers the illusion of a normal (local) procedure call while the underlying

implementation handles all network communication transparently. The typical working mechanism involves:

1. The client calls a local **client stub** procedure, passing arguments normally.
2. The client stub **marshals** (packs) the arguments into a message and sends it to the server via the underlying network/transport protocol.
3. The message is received by a **server stub**, which **unmarshals** the arguments.
4. The server stub calls the actual server procedure with the unmarshalled arguments.
5. The server procedure executes and returns a result to the server stub, which marshals the result and sends it back.
6. The client stub receives and unmarshals the result, returning it to the calling client code as if it were a normal function return value.

This entire process is hidden from the application programmer through auto-generated stub code (often from an Interface Definition Language, IDL, specification).

Limitations of RPC: (1) It cannot easily handle **partial failures** unique to distributed systems — the caller cannot always tell whether a failure occurred before or after the remote procedure executed, unlike local calls which either fully succeed or fail; (2) **pointers/references cannot be passed** directly, since the client and server have separate address spaces, requiring pass-by-value semantics or complex distributed object references; (3) it typically incurs higher and more variable **latency** compared to local calls, and (4) achieving exactly the same semantics as a local call (e.g., strict call-by-reference, identical exception behavior) is fundamentally impossible due to network delays, message loss, and independent failure of client and server.

Q5. Explain Message-Oriented Communication and Stream-Oriented Communication with their models and use cases.

Message-Oriented Communication: Processes communicate by explicitly sending and receiving discrete messages, rather than through the procedure-call abstraction of RPC. Two important models are: - **Message-Oriented Transient Communication:** Messages are only stored by the communication system while both sender and receiver are actively executing (e.g., using sockets or the Message Passing Interface, MPI); if the receiver is not running, the message is lost. - **Message-Oriented Persistent Communication:** Messages are stored (e.g., in a queue) by the communication middleware for as long as necessary until they can be delivered, allowing sender and receiver to operate independently of each other's up-time. **Message-Queuing Systems** (e.g., IBM MQSeries, or modern systems like RabbitMQ/Kafka) implement this model, decoupling producers and consumers in both time and space.

Message-oriented communication is well suited for loosely coupled, asynchronous systems — e.g., a payment processing microservice that pushes an “order placed” event to a queue, to be consumed later by an inventory service, even if that service is temporarily down.

Stream-Oriented Communication: Unlike discrete messages, some forms of communication involve a continuous flow of related data items where **timing** is as important as content — this is essential for **continuous media** such as audio and video. A stream is characterized by whether it is **simple** (single data flow, e.g., audio only) or **composite** (multiple related flows, e.g., synchronized audio + video), and by its **timing constraints**: asynchronous (no strict

timing requirement), synchronous (a maximum end-to-end delay defined), or isochronous (both a maximum and minimum delay, i.e., data must arrive within a bounded jitter window). Stream-oriented communication requires special support (e.g., Quality of Service guarantees for bandwidth and jitter) not needed for ordinary message passing, and is used in applications like video conferencing and live media streaming.

UNIT – II: Coordination, Transactions & Naming

2 Marks Questions

Q1. Why is clock synchronization needed in distributed systems? Since each machine in a distributed system has its own physical clock which can drift at different rates, clock synchronization is needed to establish a common notion of time (or ordering of events) across machines, which is essential for tasks such as timestamping distributed events, coordinating actions, and maintaining consistency.

Q2. What is Cristian's algorithm? Cristian's algorithm is a clock synchronization technique where a client requests the current time from a time server; the client estimates the round-trip delay and sets its clock to the server's returned time plus half the estimated round-trip time, compensating for network delay.

Q3. What are Logical Clocks? Name the type introduced by Lamport. Logical clocks are a mechanism to capture the “happened-before” causal ordering of events in a distributed system without relying on synchronized physical clocks. **Lamport timestamps (Lamport clocks)** are the classic type, where each process maintains a counter incremented on each event, and messages carry the sender's timestamp so the receiver can advance its own clock accordingly, ensuring that if event A causally precedes event B, then $\text{timestamp}(A) < \text{timestamp}(B)$.

Q4. What is Mutual Exclusion in distributed systems, and why is it needed? Mutual exclusion ensures that only one process at a time can access a shared resource or execute a critical section, preventing race conditions and data inconsistency, even though processes are distributed across different machines without shared memory.

Q5. Name any two distributed mutual exclusion algorithms. (1) **Centralized algorithm** — a coordinator process grants permission to enter the critical section; (2) **Token-ring algorithm** — a token is passed around a logical ring of processes, and only the process holding the token may enter the critical section. Other examples include Ricart-Agrawala's distributed algorithm and Maekawa's voting algorithm.

Q6. What is an Election Algorithm? Give one example. An election algorithm is used to select a single coordinator process among a group of distributed processes, typically the one with the highest identifier. The **Bully algorithm** is a classic example, where the process with the highest ID always “bullies” (takes over from) lower-ID processes to become coordinator when the current coordinator fails.

Q7. What is Gossip-based coordination? Gossip-based coordination (epidemic protocols) is a technique where nodes periodically exchange state information with a small number of randomly chosen peers, and this information gradually spreads (“gossips”) through the entire system, providing scalable, eventually-consistent dissemination without requiring a central coordinator.

Q8. What is a Transaction, and what does the acronym ACID stand for? A transaction is a sequence of one or more operations on shared data, treated as a single indivisible unit of work. ACID stands for **Atomicity** (all-or-nothing execution), **Consistency** (a transaction takes the system from one valid state to another), **Isolation** (concurrent transactions do not interfere with each other), and **Durability** (once committed, changes persist even after failures).

Q9. What is Serializability? Serializability is a correctness criterion for concurrent transaction execution, stating that the outcome of executing a set of transactions concurrently must be equivalent to some serial (one-at-a-time) execution of those same transactions.

Q10. Name any two weak isolation levels. (1) **Read Committed** — prevents dirty reads (reading uncommitted data from another transaction); (2) **Snapshot Isolation / Repeatable Read** — ensures a transaction sees a consistent snapshot of the database throughout its execution, preventing non-repeatable reads.

Q11. What is Flat Naming? Flat naming refers to naming schemes where names (identifiers) have no internal structure and typically must be resolved to a network address via a lookup mechanism, often implemented with techniques such as broadcasting/multicasting or Distributed Hash Tables (DHTs).

Q12. What is Structured Naming? Structured naming organizes names hierarchically (e.g., file path names, DNS domain names) using a naming graph, where names are composed of a sequence of labels that can be resolved step-by-step, typically through a directory service, to locate the named resource.

Q13. What is Attribute-Based Naming? Attribute-based naming (also called descriptive naming) identifies entities by a set of attributes (key-value pairs describing their properties) rather than a single structured or flat name, and resolution involves searching for entities matching a given set of attribute values (e.g., LDAP directories).

Q14. What is Named Data Networking (NDN)? NDN is a network architecture that names data content itself (rather than naming hosts/locations as in IP), so that data can be requested and routed/retrieved by name from any node that has a copy, enabling efficient in-network caching and content distribution.

6 Marks Questions

Q1. Explain Clock Synchronization and describe Cristian’s algorithm and the Berkeley algorithm with their limitations.

In a distributed system, each machine has an independent physical clock (typically a quartz crystal oscillator), and due to manufacturing variations and environmental factors, these clocks

drift apart over time (clock skew). Since many distributed applications (logging, transaction ordering, security certificates) rely on a consistent notion of time, clock synchronization algorithms are used to keep clocks closely aligned, either to an external accurate reference (external synchronization) or merely to each other (internal synchronization).

Cristian's Algorithm: A client process requests the current time from a time server (assumed to have an accurate reference, e.g., from a GPS or radio clock). The client measures the round-trip time (RTT) of the request-response exchange. Assuming the network delay is roughly symmetric, the client estimates the server's time at the moment of reply as $T_{\text{server}} + \text{RTT}/2$, and sets its own clock to this value. Limitation: this assumes a single reliable time server (a single point of failure) and symmetric network delay, which is often not accurate in practice; also, if RTT is large or highly variable, the synchronization accuracy suffers.

Berkeley Algorithm: Unlike Cristian's algorithm, this does not assume the existence of an accurate external time source. Instead, a coordinator process periodically polls all other machines for their local clock values, computes an average (often discarding outliers), and then sends each machine the adjustment (offset) it needs to apply to its own clock to align with this computed average. Limitation: it only achieves internal synchronization (all clocks agree with each other) but not necessarily accurate agreement with real/UTC time; also, the coordinator itself is a potential single point of failure, requiring an election mechanism if it crashes.

Both algorithms illustrate a broader challenge: perfect clock synchronization is impossible due to unpredictable network delays, so practical systems (e.g., NTP — Network Time Protocol, used widely in the Internet) combine multiple such techniques with statistical filtering to achieve reasonably accurate synchronization.

Q2. Explain Lamport's Logical Clocks and how they establish a “happened-before” relation, with an example.

Since perfectly synchronized physical clocks are difficult to achieve, Lamport proposed **logical clocks**, which capture only the relative, causal ordering of events rather than exact wall-clock time. The **happened-before relation** ($\rightarrow$) is defined as follows: (1) if events a and b occur in the same process and a occurs before b , then $a \rightarrow b$; (2) if a is the sending of a message by one process and b is the receipt of that same message by another process, then $a \rightarrow b$; (3) the relation is transitive: if $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$.

Lamport's algorithm assigns each event a logical timestamp using these rules: 1. Each process P_i maintains a counter C_i , initialized to 0. 2. Before executing any event, P_i increments its counter: $C_i \leftarrow C_i + 1$. 3. When P_i sends a message, it includes its current counter value as a timestamp in the message. 4. When P_j receives a message with timestamp T_m , it sets its own counter to $C_j \leftarrow \max(C_j, T_m) + 1$ before processing the receive event.

This ensures that if $a \rightarrow b$, then $\text{timestamp}(a) < \text{timestamp}(b)$ (though the converse is not guaranteed — two unrelated, concurrent events can end up with timestamps in either order, which is why Lamport clocks give only a **partial** causal ordering, resolved into a **total order** in practice by breaking ties using process IDs).

Example: Suppose Process P_1 has events with timestamps 1, 2. At its event with timestamp 2, P_1 sends a message to P_2 . P_2 's local counter was at 1; upon receiving the message (timestamp 2),

P2 sets its counter to $\max(1, 2) + 1 = 3$ for the receive event. This correctly reflects that the receive event happened after the corresponding send event, even though P1 and P2 have physically unsynchronized clocks.

Q3. Explain the Bully Algorithm for Coordinator Election in a Distributed System.

The Bully algorithm is used to elect a single coordinator among a group of n processes, each with a unique identifier, under the assumption that the process with the highest identifier should become coordinator, and that processes can detect the crash of the current coordinator (e.g., via timeouts).

Working: 1. When a process P notices that the current coordinator has crashed (e.g., a heartbeat/timeout expires), it initiates an election by sending an **ELECTION** message to all processes with a higher identifier than itself. 2. If P receives no response (an **OK** message) within a timeout, it concludes that it has the highest ID among the live processes and declares itself coordinator by sending a **COORDINATOR** message to all processes with lower IDs. 3. If P receives an OK message from any higher-ID process, it drops out of the election and waits for a COORDINATOR announcement, since a higher-ID process will take over the election. 4. Any process that receives an ELECTION message from a lower-ID process immediately responds with OK (to signal it is alive and will take over) and starts its own election (if it hasn't already) by sending ELECTION messages to processes with IDs higher than its own. 5. This continues until the highest-ID surviving process declares itself coordinator, and all lower-ID processes acknowledge it.

Example: Suppose processes P1, P2, P3, P4, P5 exist with P5 as the current coordinator (highest ID). If P5 crashes, and P3 is the first to notice, P3 sends ELECTION to P4 and P5. Since P5 is down, only P4 responds OK. P3 then waits, expecting P4 to become coordinator. P4, having received the ELECTION message, sends its own ELECTION to P5 (no response, since it's down), times out, and declares itself the new coordinator, announcing this to P1, P2, and P3.

The algorithm is called “Bully” because a recovering higher-ID process will always take over the coordinator role from a currently functioning lower-ID coordinator, “bullying” its way back into leadership — this is a design trade-off favoring deterministic leadership based on ID over avoiding leadership churn.

Q4. Explain ACID properties of transactions and discuss common Weak Isolation Levels and the anomalies they prevent or allow.

A **transaction** groups a sequence of read/write operations into a single logical unit that should execute reliably despite concurrency and failures, guaranteed by four properties: - **Atomicity:** All operations of a transaction either complete fully or have no effect at all (all-or-nothing); if any part fails, the entire transaction is rolled back. - **Consistency:** A transaction transforms the database from one valid (consistent) state to another, preserving any declared invariants (e.g., account balances non-negative). - **Isolation:** Concurrently executing transactions should not observe each other's intermediate, uncommitted states — ideally, execution behaves as if transactions ran one at a time (serially). - **Durability:** Once a transaction commits, its effects must survive subsequent crashes (typically ensured via write-ahead logging to persistent storage).

Serializability is the strongest, formal isolation guarantee — equivalent to some serial execution order — but it is expensive to enforce (often via locking or serializable snapshot isolation), so many systems offer **weaker isolation levels** for better performance, at the cost of allowing certain anomalies:

- **Read Uncommitted:** Allows **dirty reads** — a transaction can read data written by another transaction that has not yet committed (and might be rolled back).
- **Read Committed:** Prevents dirty reads (only committed data is visible), but still allows **non-repeatable reads** — if a transaction reads the same row twice, it may see different values if another transaction committed a change in between.
- **Repeatable Read / Snapshot Isolation:** A transaction sees a consistent snapshot of the database as of its start time, preventing non-repeatable reads, but can still suffer from **write skew** and **phantom reads** (e.g., a range query returning different rows on repeated execution, if new rows matching the condition are inserted concurrently) unless additional protections are used.
- **Serializable:** The strongest level, preventing all these anomalies by guaranteeing an execution equivalent to some serial order.

In distributed databases, achieving full serializability across nodes is particularly costly (requiring protocols such as two-phase locking or distributed consensus for commit ordering), so many real-world distributed data stores default to weaker isolation (e.g., snapshot isolation) to balance correctness with performance and availability.

Q5. Explain Flat Naming, Structured Naming, and Attribute-Based Naming with examples, and describe how name resolution works in each.

Flat Naming: Names are simple, unstructured identifiers (e.g., a random bit string or a globally unique ID) with no embedded location or hierarchy information. Resolving a flat name to a network address requires a separate lookup mechanism, since the name itself gives no clue about where the entity is. Common resolution techniques include: (1) **broadcasting/multicasting** the name to all nodes and letting the owning node respond (works for small-scale local networks), and (2) **Distributed Hash Tables (DHTs)**, such as Chord, where names are hashed to a key, and a lookup is routed through a structured overlay network of nodes, each responsible for a specific key range, resolving the name in $O(\log n)$ hops for n nodes.

Structured Naming: Names are organized hierarchically as a sequence of labels forming paths through a naming graph (e.g., directories/files in a file system, or domain names in DNS: `www.example.com`). Resolution proceeds step-by-step: starting from a root or known directory, each label is looked up to find the next directory/node, until the full path is resolved to the target entity's address. This structure naturally supports human-friendly, hierarchical organization and delegation of naming authority (e.g., different organizations manage their own DNS subdomains).

Attribute-Based Naming: Instead of a single name, entities are identified by a collection of (attribute, value) pairs describing their properties (e.g., “department=CS, role=server, location=Hyderabad”). Resolution involves querying a directory service (e.g., LDAP) for all entities whose attributes match a given search criterion, rather than resolving one specific

name to one specific entity — this supports flexible, descriptive search rather than exact-name lookup, useful when users know properties of a resource but not its exact structured/flat name.

Example contrast: To find a printer, a flat name might be a random GUID requiring a lookup table; a structured name might be /building2/floor3/printer1; an attribute-based query might be “find any color printer on floor 3 that is currently available” — each suited to different use cases depending on whether the client knows the exact identity, the exact hierarchical location, or only descriptive properties of the desired resource.

UNIT – III: Consistency, Replication, Partitioning & Consensus

2 Marks Questions

Q1. Why is replication used in distributed systems? Replication (keeping multiple copies of data on different nodes) is used to improve reliability (data survives node failures), availability (data remains accessible even if some replicas are down), and performance (clients can be served by a nearby replica, and read load can be spread across replicas).

Q2. What is a Data-Centric Consistency Model? Give one example. A data-centric consistency model defines rules about the order in which read/write operations on shared (possibly replicated) data must appear to all processes, independent of which specific client is accessing it. **Strict consistency** and **sequential consistency** are examples — sequential consistency requires that all processes see all operations in the same order, though not necessarily the real-time order.

Q3. What is a Client-Centric Consistency Model? Name one example. Client-centric consistency models provide consistency guarantees from the perspective of a single client accessing (possibly different) replicas over time, rather than guarantees across all clients simultaneously. **Eventual consistency** is a common example, where, if no new updates occur, all replicas will eventually converge to the same value, though there is no bound on how long convergence takes.

Q4. What is Leader-Follower (Leader-Based) Replication? In leader-based (primary-backup) replication, one replica is designated the leader, which accepts all write requests and propagates the changes to follower (replica) nodes; reads may be served by the leader or, for scalability, by followers (potentially returning stale data).

Q5. What is Replication Lag, and what problems can it cause? Replication lag is the delay between a write being applied at the leader and that write being applied at a follower replica. It can cause anomalies such as reading stale data immediately after a write (violating “read-your-writes” consistency) or observing writes out of order across different replicas.

Q6. What is Multi-Leader Replication? Multi-leader replication allows more than one node to accept write requests (each acting as a leader for writes), with changes propagated asynchronously between leaders; it improves write availability and can reduce latency across

geographically distributed data centers, but introduces the possibility of write conflicts that must be resolved.

Q7. What is Leaderless Replication? In leaderless replication, any replica can accept writes directly from clients (no single leader), and clients (or coordinators) send writes/reads to multiple replicas, relying on quorum-based techniques (e.g., read/write quorums) to maintain consistency, as used in systems like Amazon Dynamo/Cassandra.

Q8. What is Partitioning (Sharding)? Partitioning (also called sharding) is the technique of splitting a large dataset into smaller, disjoint subsets (partitions/shards), each stored/managed on a different node, allowing the system to scale horizontally in terms of storage and query throughput.

Q9. What is Rebalancing in the context of partitioning? Rebalancing is the process of moving data between nodes when partitions become unevenly distributed (e.g., due to node addition/removal or data growth), to restore an even distribution of load and storage across the cluster.

Q10. What is Linearizability? Linearizability is a strong consistency guarantee stating that once a write operation completes, all subsequent reads (by any client) must return that written value (or a later one) — effectively making the system behave as if there were only a single, up-to-date copy of the data, with operations appearing to take effect atomically at some point between their invocation and response.

Q11. What is the difference between Linearizability and Serializability? Linearizability is about the consistency of individual read/write operations on a single object across replicas. Serializability is about the isolation of multi-operation transactions, ensuring the combined effect of concurrently executing transactions is equivalent to some serial order. A system can provide one without the other; “strict serializability” combines both.

Q12. What is the Distributed Consensus problem? The distributed consensus problem requires a set of processes to agree on a single value (e.g., which transaction commits, or which node is the leader), despite failures or network issues, such that all non-faulty processes eventually decide on the same value.

6 Marks Questions

Q1. Explain Data-Centric Consistency Models — specifically Strict, Sequential, and Causal Consistency — with examples.

Data-centric consistency models define contracts about how a (possibly replicated) shared data store behaves when multiple processes read and write concurrently, independent of a specific client’s perspective.

- **Strict Consistency:** The strongest (largely theoretical) model — any read on a data item returns the value corresponding to the most recent write, based on true global (real) time. This requires perfectly synchronized clocks and instantaneous propagation, making it practically unachievable in real distributed systems with network delays.

- **Sequential Consistency:** Requires that the result of any execution is the same as if all operations by all processes were executed in **some sequential order**, and each process's own operations appear in that order in the sequence they were issued — but this order need not match real (wall-clock) time. Example: if process P1 writes $x=1$ then $x=2$, and process P2 later reads x , sequential consistency guarantees P2 will not see the writes out of order relative to how P1 issued them, but different processes might disagree on exactly *when* (relative to their own operations) a particular remote write becomes visible, as long as some single global ordering consistent with each process's program order exists.
- **Causal Consistency:** A weaker (more practical, scalable) model that requires only causally related operations to be seen by all processes in the same order; concurrent (causally unrelated) writes may be seen in different orders by different processes. Example: if process P1 writes a value and process P2 reads that value and then writes an update based on it, all other processes must see P1's write before P2's write (since P2's write causally depends on it) — but two independent, unrelated writes by P1 and P3 (with no causal link) can appear in different orders to different observers without violating causal consistency.

These models sit on a spectrum: strict is strongest and least practical; sequential is a widely referenced “consistency baseline”; causal consistency is weaker still but scales much better in geographically distributed systems, since it does not require global coordination for every operation — only for causally dependent ones.

Q2. Explain Leader-Follower (Single-Leader) Replication and the problems caused by Replication Lag, along with their mitigation techniques.

In **leader-follower (single-leader/primary-backup) replication**, one replica is designated as the leader; all write operations are sent to the leader, which applies them locally and then propagates the changes (via a replication log) to one or more follower replicas. Reads can be served by the leader (always up to date) or by followers (which may lag behind, but relieves read load from the leader), which is a common technique for horizontally scaling read-heavy workloads.

Because replication to followers is often asynchronous (for performance) rather than synchronous, followers can lag behind the leader, producing several observable anomalies:

1. **Read-your-writes inconsistency:** A user writes data via the leader, then immediately reads from a lagging follower and does not see their own recent write. *Mitigation:* route reads that immediately follow a write by the same user to the leader (or to a follower guaranteed to be at least as up to date as the write), for some time window.
2. **Monotonic reads violation:** A user reads from one follower and sees a value, then reads again (perhaps routed to a different, more-lagging follower) and sees an *older* value, appearing to go “back in time.” *Mitigation:* always route a given user's reads to the same replica (e.g., based on a hash of the user ID).
3. **Consistent prefix violation:** If writes happen in a causal order (e.g., a question is written before its answer), a lagging replica might show the answer before the question

is visible, due to out-of-order replication of unrelated partitions. *Mitigation*: ensure causally related writes are replicated together / assigned to the same partition, or use causality tracking.

Synchronous vs. Asynchronous replication trade-off: Fully synchronous replication (waiting for all followers to acknowledge before confirming a write) avoids these lag-related anomalies but reduces availability and increases write latency (a single slow / failed follower blocks all writes); fully asynchronous replication offers better performance/availability but risks lag-related anomalies and potential data loss if the leader fails before followers catch up. Many practical systems use a middle ground (“semi-synchronous” — waiting for acknowledgment from at least one follower).

Q3. Explain Multi-Leader and Leaderless Replication, including how each handles write conflicts.

Multi-Leader Replication: More than one node acts as a leader, independently accepting writes and asynchronously replicating them to the other leaders. This is useful in multi-datacenter deployments, where each datacenter has its own local leader to serve local write traffic with low latency, and changes are replicated across datacenters in the background. The major challenge is **write conflicts** — if two leaders concurrently accept different updates to the same data item (e.g., two users editing the same document offline, later syncing), the system must detect and resolve this. Common conflict-resolution strategies include: **Last-Write-Wins (LWW)** (using timestamps to pick a “winner,” at the risk of silently discarding one update), **conflict-free replicated data types (CRDTs)** (data structures designed so concurrent updates can be merged deterministically without conflict, e.g., counters or sets), and **application-level / manual conflict resolution** (surfacing the conflict for a human or business-logic decision, as in version control systems).

Leaderless Replication: No single node is designated leader; any replica can accept read and write requests directly from clients. A common technique is **quorum-based replication**: with N total replicas, a write is considered successful once acknowledged by W replicas, and a read queries R replicas and returns the most recent value among the responses (often determined via version numbers/vector clocks); consistency is improved if $W + R > N$, ensuring read and write quorums always overlap in at least one replica. Conflicts between concurrent writes are typically detected using **version vectors** and resolved either automatically (e.g., LWW) or by returning multiple conflicting versions (“siblings”) to the application/client for resolution, as in Amazon’s Dynamo system and its descendants like Cassandra and Riak. Leaderless replication offers high write availability (since no single leader is a bottleneck or single point of failure), at the cost of more complex conflict handling and only eventual, not immediate, consistency guarantees in general.

Q4. Explain Partitioning (Sharding) strategies for key-value data and the problem of Rebalancing.

Partitioning splits a large dataset across multiple nodes so that no single node needs to store or serve the entire dataset, enabling horizontal scalability. Two common partitioning strategies for key-value data are:

1. **Partitioning by Key Range:** Keys are sorted, and contiguous ranges of keys are assigned to different partitions/nodes (e.g., keys A–D on node 1, E–H on node 2, etc.). This makes range queries (e.g., “find all keys between X and Y”) efficient, since related keys are stored together, but risks **hot spots** if certain key ranges are accessed much more frequently than others (e.g., all writes to keys starting with today’s date landing on one partition).
2. **Partitioning by Hash of Key:** A hash function is applied to each key, and the resulting hash value determines which partition the key belongs to (e.g., using consistent hashing). This spreads keys much more evenly across partitions, avoiding hot spots, but sacrifices efficient range queries, since consecutive keys are scattered across different, unrelated partitions.

Partitioning and Secondary Indexes: When a query needs to filter by a field other than the primary partition key (a secondary index, e.g., searching for all users in a particular city), two approaches exist: **document-partitioned (local) secondary indexes**, where each partition maintains its own secondary index covering only its own data (requiring a scatter-gather query across all partitions), and **term-partitioned (global) secondary indexes**, where the secondary index itself is partitioned by the indexed term/value (making reads for a specific value efficient from one partition, but making writes more expensive since a single write may need to update index entries on a different partition/node).

Rebalancing: As data grows, nodes are added/removed, or access patterns skew, partitions can become unevenly distributed, requiring **rebalancing** — moving partitions (or splitting/merging them) between nodes to restore balance. Good rebalancing strategies (1) avoid unnecessary data movement (moving only what’s needed), (2) keep the system available/operational during rebalancing, and (3) can be automated but often benefit from some human oversight, since fully automatic rebalancing can occasionally trigger cascading failures under unexpected conditions (e.g., a node appearing overloaded and triggering unnecessary further rebalancing).

Q5. Explain Linearizability and its relationship to Consensus, and describe how Distributed Transactions relate to the Consensus problem.

Linearizability is a strong consistency model for a single data object/register: it requires that all operations (reads and writes) from all clients appear to take effect **atomically** at some single point in time between their invocation and completion, consistent with a real-time ordering — i.e., once a write completes, every subsequent read (by any client, on any replica) must reflect that write or a later one. This makes a distributed, possibly-replicated system behave, from the outside, exactly like a single, non-replicated copy of the data, greatly simplifying reasoning for applications (e.g., ensuring a uniqueness constraint, or a lock, behaves correctly), at the cost of requiring coordination (and hence higher latency, and reduced availability during network partitions, per the CAP theorem) for every operation.

Relationship to Consensus: Implementing linearizable operations on a replicated object is formally equivalent to solving the distributed **consensus** problem — all replicas must agree, for each operation, on a single, globally-ordered outcome, exactly what consensus algorithms (like Paxos or Raft) are designed to guarantee. This is why linearizable systems are typically

built on top of a consensus protocol used to elect a leader and agree on the order of operations in a replication log.

Distributed Transactions and Consensus: A distributed transaction spans multiple nodes (e.g., updating data on node A and node B atomically). Ensuring atomicity across nodes (all commit, or all abort) is itself a consensus problem — this is classically solved using the **Two-Phase Commit (2PC)** protocol, where a coordinator first asks all participants to “prepare” (vote to commit or abort), and only if all vote to commit does the coordinator send a final “commit” decision (otherwise, “abort”). However, 2PC is blocking — if the coordinator crashes after the prepare phase but before sending the final decision, participants must block, holding locks, until the coordinator recovers, which is a significant availability weakness. More robust solutions use **consensus algorithms (Paxos/Raft)** to replicate the coordinator’s decision itself, so that even if the original coordinator crashes, a replacement can be elected and safely determine (or re-derive) the outcome, avoiding indefinite blocking — illustrating how distributed transactions, replication, and consensus are deeply interconnected problems in distributed systems design.

UNIT – IV: Faults, Unreliability & Fault Tolerance

2 Marks Questions

Q1. What is a Partial Failure, and how does it differ from failures in non-distributed systems? A partial failure occurs when some components of a distributed system fail while others continue working correctly — unlike a single (non-distributed) machine, which typically either works entirely or crashes entirely. Partial failures are nondeterministic and much harder to detect/handle, since a remote node’s silence could mean it crashed, is slow, or the network dropped the message.

Q2. List any three ways a network can be unreliable. (1) Messages can be **lost** (dropped by the network); (2) messages can be **delayed** arbitrarily (variable/unbounded latency); (3) messages can be **duplicated** or **reordered** (arrive out of the order they were sent, or arrive more than once).

Q3. What is a Timeout, and what is the fundamental problem with using timeouts to detect failures? A timeout is a fixed period after which, if no response is received, a process assumes the remote process/message has failed. The fundamental problem is that a timeout cannot distinguish between an actual crash and a slow node/network — a longer timeout increases detection delay, while a shorter timeout increases false-positive failure detections.

Q4. What are the two types of clock problems discussed in distributed systems? (1) **Clock drift** — clocks on different machines tick at slightly different rates, causing them to diverge over time even if initially synchronized; (2) **Clock jumps** — clocks can suddenly jump forward or backward (e.g., due to NTP corrections or leap seconds), violating the assumption that time always moves forward monotonically.

Q5. What is meant by “Knowledge, Truth, and Lies” in the context of distributed systems? This refers to the philosophical/practical challenge that in a distributed system, a node cannot be certain about the true state of other nodes or the network — it can only reason based on the (possibly incomplete, delayed, or false) information it receives, and must design algorithms (e.g., using quorums, fencing tokens) that remain correct even when some nodes provide incorrect or stale information (whether due to failure or, in adversarial settings, malicious behavior — “Byzantine” faults).

Q6. What is Process Resilience, and how is it typically achieved? Process resilience is the ability of a distributed system to continue functioning correctly despite the failure of some of its component processes. It is typically achieved through **replication of processes into groups**, so that if one member of the group fails, others can continue providing the service.

Q7. What is a Byzantine Fault? A Byzantine fault is a failure in which a component behaves arbitrarily or maliciously (e.g., sending conflicting or false information to different parts of the system) rather than simply crashing or being silent, making it the most general and difficult class of faults to tolerate.

Q8. What is Reliable Group Communication? Reliable group communication ensures that a message sent to a group of processes is delivered to all non-faulty members of the group, often with additional guarantees such as consistent ordering across all members (e.g., all members deliver messages in the same order).

Q9. What is Distributed Commit, and name one protocol used for it. Distributed commit ensures that a set of distributed processes/nodes participating in a transaction either all commit or all abort, maintaining atomicity across multiple nodes. The **Two-Phase Commit (2PC)** protocol is a classic solution.

Q10. What is Checkpointing in the context of fault recovery? Checkpointing is the technique of periodically saving a process’s state to stable storage so that, after a crash, the process can be restarted from the most recent checkpoint rather than from the very beginning, reducing the amount of lost work/recomputation.

6 Marks Questions

Q1. Explain the challenges posed by Partial Failures and Unreliable Networks in Distributed Systems, and discuss why detecting failures reliably is fundamentally difficult.

Unlike a single computer, where a program either runs correctly or the whole machine crashes (a fail-stop model that is relatively easy to detect and reason about), a distributed system is inherently susceptible to **partial failures** — some nodes may crash, some may be slow, and some may be unreachable due to network issues, all while other parts of the system continue operating normally. This nondeterminism means the same distributed program might work perfectly on one run and fail unpredictably on another, depending on the timing of failures and messages.

Compounding this, the **network itself is unreliable**: messages sent between nodes can be **lost** (e.g., a router drops a packet), **arbitrarily delayed** (queuing delays, congestion, or even network partitions that isolate groups of nodes from each other for extended periods), **duplicated**, or **delivered out of order**. Since a process cannot directly observe the internal state of a remote process — it can only infer things based on whether and when messages/responses arrive — a fundamental difficulty arises: **if a process does not receive an expected response, it cannot definitively determine whether (a) the remote process has crashed, (b) the remote process is just slow (e.g., under heavy load or GC pause), (c) the request message was lost in transit, or (d) the response message was lost in transit**. All four scenarios look identical from the requester's point of view.

The standard practical solution is to use **timeouts**: if no response arrives within some time limit, assume failure. But this is an imperfect heuristic with an inherent trade-off — a short timeout detects failures quickly but risks many **false positives** (declaring a merely slow node “dead,” potentially triggering unnecessary and costly failover actions, or worse, having two nodes simultaneously believe they are the leader — a “split-brain” scenario); a long timeout reduces false positives but delays legitimate failure detection, during which the system may be unavailable or inconsistent. This is why systems favor **explicit acknowledgments, retries with idempotent operations, and quorum-based decisions** rather than relying purely on a single timeout-based judgment from a single observer, and why formal distributed algorithms are typically designed under an assumption of an **asynchronous or partially synchronous network model** (bounded but unknown delays) rather than assuming reliable, bounded-delay delivery.

Q2. Explain the problem of Unreliable Clocks in distributed systems and discuss its implications for ordering events and for time-based decisions.

Distributed systems often rely on clocks for two different purposes: (1) measuring **durations/timeouts** (relative time) using monotonic clocks, and (2) obtaining the current **time-of-day** (wall-clock time), often synchronized approximately via NTP. Both are unreliable in different ways:

- **Clock drift**: Every machine's local hardware clock ticks at a very slightly different rate due to manufacturing variance and temperature, so even if two clocks start perfectly synchronized, they will gradually drift apart (typically on the order of milliseconds per day, though it can be worse under certain conditions), meaning a raw comparison of two nodes' timestamps cannot be trusted to be precisely accurate.
- **Clock jumps**: Wall-clock time can be forcibly adjusted — e.g., an NTP synchronization step correcting a clock that has drifted too far, a manual administrator change, or leap-second adjustments — potentially causing the clock to jump forward or even **backward** in time. If application code (e.g., a lock's expiry check, or an event ordering system) assumes wall-clock time is monotonically increasing, such a jump can cause serious correctness bugs — e.g., a lease/lock that should have expired appears to still be valid (or a valid lease appears already expired), leading to conflicting concurrent access.

Implications: Because of these issues, using wall-clock timestamps directly to determine the order of events across different machines (e.g., “last-write-wins” conflict resolution based on

timestamps) is risky, since clock skew can cause an event that actually happened later to have an *earlier* recorded timestamp than one that happened earlier, silently discarding the wrong update. Best practices in distributed systems include: using **logical clocks** (e.g., Lamport clocks, vector clocks) to establish causal order instead of relying on physical time where correctness matters; using **monotonic clocks** (which never jump backward) specifically for measuring elapsed durations/timeouts rather than the wall-clock/time-of-day clock; and, for time-sensitive coordination such as distributed locks, using techniques like **fencing tokens** (a monotonically increasing number issued with each lock grant, which downstream systems can use to reject stale/out-of-order requests) rather than trusting clock-based expiry alone.

Q3. Explain Process Resilience and the design considerations for organizing process groups to tolerate failures.

Process resilience is achieved by replicating a critical service across a **group of processes**, so the group as a whole can survive the failure of individual members. Key design considerations include:

1. **Flat vs. Hierarchical Groups:** In a **flat group**, all members are equal peers, and decisions (e.g., which operation to perform next) are typically made through voting/consensus among all members — this offers good fault tolerance (no single point of failure) and symmetry, but decision-making can be slower due to the need for coordination among many peers. In a **hierarchical group**, one member acts as a coordinator/leader making decisions and disseminating them to the rest — this is simpler and faster (no need for group-wide voting on every decision) but reintroduces a single point of failure (the coordinator) unless combined with a leader-election mechanism to promote a new coordinator upon failure.
2. **Group Membership Management:** The system needs a reliable way to track which processes currently belong to the group (join/leave/crash), since operations like reliable multicast delivery and quorum calculations depend on knowing the current, agreed-upon membership — this itself is a form of the consensus problem (all members must agree on the current membership view).
3. **Failure Masking via Replication Degree (k-fault tolerance):** To tolerate up to k simultaneous crash failures using simple replication and majority voting, a group typically needs at least $2k+1$ members (so that a majority of $(k+1)$ can still be reached even if k members fail). Tolerating **Byzantine (arbitrary/malicious) failures**, rather than simple crashes, requires an even higher replication degree — at least $3k+1$ members — since the system must be able to distinguish correct majority responses even when up to k members actively send incorrect or conflicting information.
4. **Consistent State Replication:** Beyond simply having multiple copies of a process, resilience requires that all replicas process the same sequence of inputs/requests in the same order (state machine replication) so their internal states remain consistent — this is why reliable, ordered group communication and/or a consensus protocol (like Raft) underlies most practical process-resilience implementations.

Q4. Explain the Two-Phase Commit (2PC) protocol for Distributed Commit, including its phases, and discuss its main limitation.

Distributed transactions that span multiple nodes (e.g., a transaction that must debit an account on node A and credit an account on node B) require all participating nodes to agree on the same outcome — either all commit their part, or all abort — to preserve atomicity. **Two-Phase Commit (2PC)** is the classic protocol for this:

Phase 1 — Voting/Prepare Phase: 1. A designated **coordinator** sends a **PREPARE** (or “vote-request”) message to all participant nodes involved in the transaction. 2. Each participant performs the transaction’s operations tentatively (without making them permanently visible), writes them to a durable log, and replies to the coordinator with a **vote: YES (ready to commit)** if it can guarantee it will be able to commit, or **NO (abort)** if it cannot (e.g., due to a local constraint violation or failure).

Phase 2 — Commit/Abort (Decision) Phase: 1. If the coordinator receives **YES** votes from *all* participants, it decides to **commit**, and sends a **COMMIT** message to all participants; each participant then makes its tentative changes permanent and releases any locks held. 2. If the coordinator receives even a single **NO** vote (or a timeout from any participant), it decides to **abort**, and sends an **ABORT** message to all participants, causing them to roll back their tentative changes. 3. Once a participant that voted YES receives the final decision, it durably records the outcome and acknowledges completion to the coordinator.

Main Limitation — Blocking on Coordinator Failure: The critical weakness of 2PC is that after a participant votes **YES** in Phase 1, it must hold its locks and wait indefinitely for the coordinator’s final decision — if the coordinator crashes at this point (after collecting votes but before, or while, sending the final decision), the participant **cannot unilaterally decide** whether to commit or abort (since another participant may have voted NO, or the coordinator may have already decided commit and only failed to notify everyone), so it must **block**, potentially holding locks on shared resources for an extended period until the coordinator recovers. This makes 2PC’s availability fundamentally dependent on the coordinator’s own reliability — a significant drawback addressed in practice by making the coordinator itself fault-tolerant (e.g., by replicating the coordinator’s decision using a consensus protocol like Paxos, so a new coordinator can safely take over and determine the correct outcome, an approach sometimes called Three-Phase Commit or Paxos Commit).

Q5. Explain Reliable Client-Server Communication and Reliable Group Communication, and describe the guarantees they aim to provide.

Reliable Client-Server Communication: In basic RPC, a client sends a request and waits for a response, but as discussed, network/process failures create ambiguity about what actually happened if no response arrives. To provide reliability guarantees, systems typically define different **call semantics**:

- **At-most-once semantics:** The client makes no retry attempt; the remote operation is executed either zero times (if the request/response was lost, or the server crashed) or exactly once — simple, but risks the operation simply never happening.
- **At-least-once semantics:** The client retries the request (possibly multiple times) until it receives a response, guaranteeing the operation is attempted at least once, but risking it being executed **multiple times** if earlier responses were lost even though the server did process the request (unless the server can detect and ignore duplicate requests).
- **Exactly-once semantics:** The ideal but hardest-to-achieve guarantee — the operation is guaranteed to have taken effect exactly once despite any combination of message loss, duplication, or crashes. It is typically

approximated in practice by combining at-least-once retries (idempotent by design, or deduplicated using unique request identifiers) with server-side deduplication logic, since true exactly-once delivery is provably impossible to guarantee in a fully general asynchronous network with arbitrary failures.

Reliable Group Communication: When a message must be delivered to an entire group of (replicated) processes rather than a single server, additional guarantees are typically layered on top of basic (unreliable) multicast: - **Reliable multicast:** Guarantees that a message sent to the group is eventually delivered to every non-faulty member of the group (not just some of them), typically implemented using techniques like positive/negative acknowledgments and retransmission among group members. - **Atomic multicast:** Extends reliable multicast with an ordering guarantee — all members of the group must deliver all messages in the **same order**, which is critical for keeping replicated state machines consistent (if replicas processed updates in different orders, their states would diverge). Atomic multicast is, in fact, formally equivalent in difficulty to solving distributed consensus, and virtual synchrony models (which tie message delivery guarantees to a well-defined, agreed group membership view) are a classic approach used to implement it correctly even as group membership changes due to failures or new members joining.

Together, these reliable communication abstractions let application developers build correct distributed services (e.g., replicated databases, coordination services like ZooKeeper) without needing to re-solve the underlying failure-handling and ordering challenges from scratch for every application.

UNIT – V: MapReduce & Distributed File Systems

2 Marks Questions

Q1. What is MapReduce? MapReduce is a programming model and associated execution framework for processing large datasets in parallel across a cluster of machines, using two user-defined functions: a **Map** function that processes input key-value pairs to generate intermediate key-value pairs, and a **Reduce** function that aggregates/combines all intermediate values associated with the same key.

Q2. What is the role of the “Shuffle” phase in MapReduce? The shuffle phase is the intermediate step between Map and Reduce where the framework redistributes (sorts and transfers) all intermediate key-value pairs produced by the mappers so that all values for a given key are grouped together and sent to the same reducer.

Q3. What is Apache Hadoop? Apache Hadoop is an open-source framework that implements the MapReduce programming model along with the Hadoop Distributed File System (HDFS), enabling distributed storage and parallel processing of very large datasets across clusters of commodity hardware.

Q4. What is HDFS, and what is its typical block size? HDFS (Hadoop Distributed File System) is a distributed file system designed to store very large files reliably across many

machines by splitting files into large fixed-size blocks (typically 128 MB, though configurable) and replicating each block (typically 3 copies) across different nodes for fault tolerance.

Q5. What is a NameNode and a DataNode in HDFS? The **NameNode** is the master server that manages the file system namespace/metadata (which blocks make up which files, and their locations), while **DataNodes** are the worker servers that actually store the data blocks and serve read/write requests from clients.

Q6. What is Amazon Elastic MapReduce (EMR)? Amazon EMR is a managed cloud service that provisions and runs Hadoop, Spark, and other big-data frameworks on Amazon Web Services (AWS) infrastructure, removing the need to manually set up and manage a physical cluster.

Q7. What is Apache Pig? Apache Pig is a high-level data-flow scripting platform (using a language called Pig Latin) that compiles data-processing scripts into a series of MapReduce jobs, making it easier to write complex ETL/data-transformation pipelines without writing low-level Map/Reduce Java code directly.

Q8. What is Apache Hive? Apache Hive is a data warehouse infrastructure built on top of Hadoop that provides an SQL-like query language (HiveQL), allowing analysts to query large datasets stored in HDFS using familiar SQL syntax, which Hive translates internally into MapReduce (or other execution engine) jobs.

Q9. What is a Combiner function in MapReduce? A combiner is an optional, local “mini-reduce” function that runs on the output of each mapper before the shuffle phase, pre-aggregating intermediate values with the same key on the same node to reduce the volume of data transferred across the network during the shuffle.

Q10. Name any two limitations of the classic MapReduce model that motivated “Beyond MapReduce” systems. (1) MapReduce is poorly suited for **iterative algorithms** (e.g., machine learning, graph algorithms) since each iteration requires a full read/write cycle to disk between MapReduce jobs, causing high overhead; (2) It is designed primarily for **batch processing** and does not naturally support low-latency, interactive, or streaming/real-time data processing, motivating systems like Apache Spark (in-memory processing) and stream-processing frameworks (e.g., Apache Flink, Kafka Streams).

6 Marks Questions

Q1. Explain the MapReduce programming model with its phases, and illustrate with a Word Count example.

The MapReduce programming model processes large datasets by decomposing the computation into two user-defined functions operating on key-value pairs, executed in parallel across a cluster:

1. **Map phase:** The input dataset is split into chunks and distributed across worker nodes. Each mapper applies the user-defined `map(key, value)` function to its chunk of input, producing a set of intermediate key-value pairs.

2. **Shuffle and Sort phase:** The framework automatically redistributes the intermediate key-value pairs across the cluster so that all pairs sharing the same key end up at the same reducer node, and sorts them by key at each reducer.
3. **Reduce phase:** Each reducer applies the user-defined `reduce(key, list_of_values)` function to the values grouped under each key, producing the final output (typically also key-value pairs).

Word Count Example: Suppose we want to count the occurrences of each word across a large collection of text documents.

- **Map function:** For each input document (key = document ID or offset, value = line of text), the mapper tokenizes the text into words and emits an intermediate pair (word, 1) for every occurrence of every word. For example, given the input line “the cat sat on the mat,” the mapper emits: (the,1), (cat,1), (sat,1), (on,1), (the,1), (mat,1).
- **Shuffle:** The framework groups all intermediate pairs by key (word), so all the (the, 1) pairs from across the entire dataset are routed to the same reducer, along with all (cat, 1) pairs, etc.
- **Reduce function:** For each word key, the reducer receives a list of all its counts (e.g., the → [1,1,1,...]) and sums them: `reduce(the, [1,1,1]) → (the, 3)`. The final output is a list of (word, total_count) pairs for the entire input corpus.

This model’s power lies in its simplicity and automatic parallelization/fault tolerance: the framework (e.g., Hadoop) handles splitting data, scheduling mappers/reducers across the cluster, re-running failed tasks on other nodes, and managing the shuffle, while the programmer only needs to write the map and reduce logic.

Q2. Explain how MapReduce achieves Scaling and Fault Tolerance when processing very large datasets on commodity hardware.

Scaling: MapReduce achieves horizontal scalability primarily through **data parallelism** — since the map function is applied independently to each chunk (split) of the input data, and there is no dependency between different mappers, the framework can run as many mapper tasks in parallel as there are available worker machines/cores, and adding more machines to the cluster directly increases throughput (near-linear scaling for the map phase, assuming data is evenly distributed). Similarly, the reduce phase parallelizes across the distinct keys, with each reducer independently processing its assigned subset of keys. To further reduce network overhead during the shuffle phase, an optional **combiner** function performs partial aggregation locally on each mapper’s output before it is transferred over the network, and the framework schedules mapper tasks preferentially on (or near) the nodes that already store the relevant input data (“data locality” optimization, exploiting HDFS’s block placement), minimizing data movement across the network — a key design principle given that in large clusters, network bandwidth is often more constrained than local disk/compute.

Fault Tolerance: Because commodity hardware clusters (potentially thousands of machines) experience frequent individual node failures, MapReduce frameworks (like Hadoop) are

designed assuming failures are the norm rather than an exception: - The **master node** (JobTracker/ResourceManager in Hadoop) monitors the health of worker nodes via periodic heartbeats; if a worker fails to report within a timeout, it is considered dead. - Any map or reduce **tasks that were running on a failed worker are automatically re-scheduled and re-executed on a different, healthy worker**, since map and reduce tasks are designed to be idempotent/re-runnable (as long as inputs remain available, e.g., through HDFS's own replication). - Because intermediate map output is typically written to local disk (not just memory) before being shuffled, a reducer can re-fetch this data even if the original mapper task is later relaunched elsewhere, and completed task outputs are not considered final until acknowledged, preventing partial/corrupted results from silently propagating. - The underlying distributed file system (HDFS) itself replicates each data block (typically 3x) across different nodes/racks, so that even permanent node failures do not cause data loss, and MapReduce tasks can always find an alternate healthy replica to read their required input from.

Q3. Explain the architecture of the Hadoop Distributed File System (HDFS), including the roles of the NameNode and DataNodes.

HDFS is designed to reliably store very large files across a cluster of commodity machines, optimized for high-throughput, write-once/append, sequential-read access patterns (rather than low-latency random access, which is a deliberate trade-off for simplicity and throughput at scale).

Architecture: - Each file is split into large, fixed-size **blocks** (default typically 128 MB, much larger than typical local file system block sizes, chosen to minimize the overhead of managing metadata for very large files and to make sequential I/O efficient). - Each block is **replicated** (by default, 3 copies) across different DataNodes, and HDFS's default placement policy tries to spread replicas across different physical racks, so that even the failure of an entire rack (e.g., due to a shared power/network failure) does not cause data loss.

NameNode (Master): A single, central server that manages the file system's **namespace and metadata** — the directory structure, file-to-block mappings, and the current locations (which DataNodes hold which block replicas). It does not store the actual file data itself, only metadata (kept in memory for fast access, with a persistent edit log and periodic checkpoint image for durability/recovery). Because the NameNode is central to every file system operation (lookups, creating new files, etc.), it is historically a **single point of failure** — addressed in modern Hadoop deployments through **High-Availability (HA) NameNode** configurations, which maintain a standby NameNode kept in sync (e.g., via a shared edit log in a system like a Quorum Journal Manager) that can take over automatically if the active NameNode fails.

DataNodes (Workers): These are the many worker servers that actually store the physical data blocks on their local disks, and directly serve read/write requests for block data to/from clients (after the client has first asked the NameNode which DataNodes hold the relevant blocks). DataNodes periodically send **heartbeats and block reports** to the NameNode, both to signal that they are alive and to report which blocks they currently hold; if a DataNode's heartbeat is not received within a timeout, the NameNode marks it as dead and initiates **re-replication** of the blocks that were stored on it (using the remaining healthy replicas) to restore the desired replication factor elsewhere in the cluster.

This master-worker split — a lightweight, memory-resident metadata service (NameNode) combined with many independent, simple block-storage servers (DataNodes) — allows HDFS to scale to very large clusters storing petabytes of data while remaining reasonably simple to reason about and, with HA configurations, reasonably fault-tolerant despite the metadata service's originally centralized design.

Q4. Compare Apache Pig and Apache Hive as higher-level abstractions over MapReduce, explaining their programming models and typical use cases.

Writing raw MapReduce jobs directly (in Java) for every data-processing task is verbose and requires expressing even simple operations (filtering, joining, grouping) as explicit map/reduce logic. **Pig** and **Hive** were developed to raise the level of abstraction, letting users express data-processing logic more declaratively, which the respective system then compiles down into a series of underlying MapReduce (or other execution engine) jobs automatically.

Apache Pig: Provides a **procedural, data-flow scripting language called Pig Latin**, where a script describes a sequence of data transformation steps (LOAD, FILTER, JOIN, GROUP, FOREACH...GENERATE, STORE, etc.) applied to datasets, closer in style to writing a data pipeline/ETL script step-by-step than to declarative SQL. Pig is particularly well-suited for **exploratory data analysis and complex, multi-step ETL (Extract-Transform-Load) pipelines**, where the sequence of transformations is naturally expressed as an ordered series of operations, and where the input data may be semi-structured or not require a fixed schema upfront.

Apache Hive: Provides a **declarative, SQL-like query language called HiveQL**, which allows users (especially those already familiar with relational databases/SQL) to define table schemas over data stored in HDFS and write familiar SELECT/WHERE/JOIN/GROUP BY-style queries, which Hive's query planner/optimizer compiles into an execution plan of underlying jobs. Hive is particularly well-suited for **structured data warehousing and ad hoc analytical/BI queries** over large datasets by users who prefer (or whose tools require) a SQL interface, without needing to write procedural transformation scripts.

Comparison: Pig's data-flow style gives more explicit, step-by-step control over the exact sequence of transformations (useful for complex pipelines), whereas Hive's declarative SQL style is easier to use for straightforward analytical queries and integrates naturally with existing SQL-based BI/reporting tools; in practice, organizations often use both — Pig for building ETL pipelines that prepare/clean raw data, and Hive for enabling analysts to query the resulting cleaned, structured data using familiar SQL.

Q5. Explain the limitations of the classic MapReduce model and discuss “Beyond MapReduce” systems (such as Apache Spark) that address them.

While MapReduce popularized large-scale distributed data processing on commodity clusters, its rigid two-phase (map-then-reduce) model, combined with its reliance on writing intermediate results to disk between every job, exposes several limitations:

1. **High latency for iterative algorithms:** Many important workloads — iterative machine learning algorithms (e.g., k-means clustering, gradient descent), and graph algorithms (e.g., PageRank) — require repeatedly processing the same dataset over

many rounds/iterations. In classic MapReduce, each iteration is a separate MapReduce job that must read its input from disk (typically HDFS) and write its output back to disk, even though most of the data may be unchanged between iterations — this disk I/O overhead, repeated at every iteration, makes iterative workloads very slow.

2. **Poor support for interactive/ad hoc queries:** Because every MapReduce job incurs significant startup overhead (scheduling tasks, reading from disk) and typically takes at least tens of seconds to complete even for small jobs, MapReduce is a poor fit for interactive data exploration where an analyst expects near-immediate query responses.
3. **No native support for streaming data:** MapReduce is fundamentally a **batch-processing** model, operating on a fixed, complete input dataset; it does not naturally support continuously arriving data (e.g., real-time log or sensor data) where results are needed with low latency as new data arrives, rather than after collecting a complete batch.

Apache Spark, the most prominent “Beyond MapReduce” system, directly addresses limitation (1) and (2) through its core abstraction, the **Resilient Distributed Dataset (RDD)** — an immutable, distributed collection of objects that can be cached **in-memory** across a cluster and reused across multiple operations/iterations, avoiding the need to write intermediate results back to disk between steps. This makes Spark dramatically faster (often 10-100x) than classic MapReduce for iterative and interactive workloads, while retaining fault tolerance through RDD “lineage” (Spark can recompute a lost partition of data by re-applying the recorded sequence of transformations that produced it, rather than needing physical replication of every intermediate result). Spark also provides a much richer, more expressive set of built-in operations (map, filter, join, groupBy, and many others) directly usable from a single unified API, rather than requiring every task to be manually decomposed into just “map” and “reduce” steps.

Beyond Spark, dedicated **stream-processing frameworks** (e.g., Apache Flink, Kafka Streams, Spark Structured Streaming) directly address limitation (3) by processing data as a continuous, unbounded stream of events with low-latency, incremental computation (rather than only supporting fixed, complete batches), enabling real-time analytics use cases (e.g., real-time fraud detection, live dashboards) that classic batch-oriented MapReduce cannot practically support.

End of Question Bank — Prepared as per Osmania University B.E. AI/ML Distributed Systems (PC 603 CS) syllabus, with reference to Van Steen & Tanenbaum’s “Distributed Systems” (PHI, 4th Ed., 2023) and Martin Kleppmann’s “Designing Data-Intensive Applications” (O’Reilly, 2017).